

Тема роботи:

АРХІТЕКТУРНЕ МОДЕЛЮВАННЯ СИСТЕМИ

Мета роботи: розробка архітектури системи.

Завдання роботи: освоїти прийоми проектування архітектури системи і необхідних класів компонентів.

Зміст роботи:

- 1) виділення архітектурних рівнів;
- 2) моделювання розподіленої конфігурації системи;
- 3) створення діаграми розміщення;
- 4) проектування класів;
- 5) створення діаграми станів.

Приклад виконання лабораторної роботи

Проектування системи

1. Проектування архітектури

Цілі проектування архітектури системи:

- аналіз взаємодій між класами аналізу, виявлення підсистем і інтерфейсів;
- уточнення архітектури з урахуванням можливостей повторного використання;
- ідентифікація архітектурних рішень і механізмів, необхідних для проектування системи.

Вводяться глобальні пакети:

- базисні (foundation) класи (списки, черги і так далі);
- обробники помилок (error handling classes);
- математичні бібліотеки;
- утиліти;
- бібліотеки інших постачальників.

Визначаються проектні класи (design classes):

- клас аналізу відображається в проектний клас, якщо він простий або представляє єдину логічну абстракцію;
- складний клас аналізу може бути розбитий на декілька класів, перетворений в пакет або в підсистему.

Приклади можливих підсистем:

- класи, що забезпечують складний комплекс послуг (наприклад, забезпечення безпеки і захист);
- граничні класи, що реалізують складний призначений для користувача інтерфейс або інтерфейс із зовнішніми системами;
- різні продукти: комунікаційне ПЗ (middleware, підтримка COM/CORBA), доступ до баз даних, типи і структури даних (стеки, списки, черги), загальні утиліти (математичні бібліотеки), різні прикладні продукти.

Ухвалення рішення про перетворення класу в підсистему визначається досвідом і знаннями архітектора проекту.

Угоди з проектування інтерфейсів:

1. Ім'я інтерфейсу: коротке (одно-два слова), відбиваюче його роль в системі.
2. Опис інтерфейсу: повинно відображати його обов'язки (розмір – невеликий абзац).

3. Опис операцій: ім'я, що відображати результат операції, ключові алгоритми, значення, що повертається, параметри з типами.

4. Документування інтерфейсу: характер використання операцій і порядок їх виконання (показується за допомогою діаграм послідовності), тестові плани і сценарії і так далі. Уся ця інформація об'єднується в спеціальний пакет із стереотипом <<subsystem>>, який містить елементи, що утворюють підсистему, діаграми послідовності і/або кооперативні діаграми, що описують взаємодію елементів при реалізації операцій інтерфейсу, і інші діаграми.

5. Клас <<subsystem proxy>> безпосередньо реалізує інтерфейс і управляє реалізацією його операцій.

6. Усі інтерфейси мають бути повністю визначені в процесі проектування архітектури, оскільки вони служитимуть в якості точок синхронізації при паралельній розробці.

Виділення архітектурних рівнів:

- Application Layer – містить елементи прикладного рівня (призначений для користувача інтерфейс);

- Business Services Layer – містить елементи, що реалізують бізнес-логіку додатків (найбільш стійка частина системи);

- Middleware Layer – забезпечує сервіси, незалежні від платформи.

Приклад виділення архітектурних рівнів та об'єднання елементів моделі в пакети для системи реєстрації наведено на рисунку 2.22.

Для того, щоб помістити клас в пакет, досить просто перетягнути його в браузері на потрібний пакет.

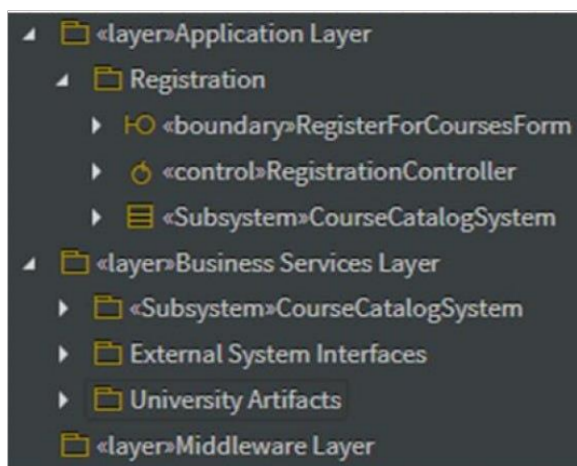


Рисунок 2.22 – Виділення архітектурних рівнів

Дане подання відображає наступні рішення, прийняті архітектором.

Виділено три архітектурних рівня (створені три пакети зі стереотипом << layer >>):

- треба створити три пакети в Logical View > Design Model: Application Layer, Business Services Layer, Middleware Layer;
- в пакеті Application Layer створити пакет Registration, куди включені граничні і керуючі класи (граничний клас CourseCatalogSystem перетворений в підсистему, змінити стереотип на << subsystem >>):

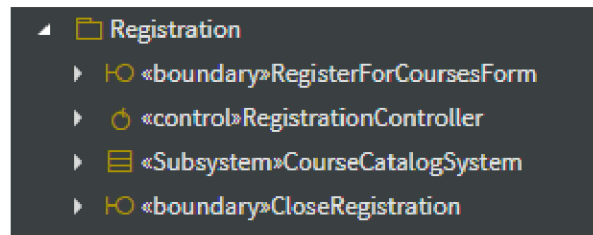


Рисунок 2.23 – Дерево Registration Layer

- в пакет Business Services Layer, крім підсистеми CourseCatalogSystem (створити новий пакет, а в ньому граничний клас з такою ж назвою), включені ще два пакети: пакет External System Interfaces, який включає в себе новий об'єкт «Interface» (Add > Interface) з назвою ICourseCatalogSystem, а пакет University Artifacts – всі класи-сутності.

Структура архітектури на рисунку 2.24.

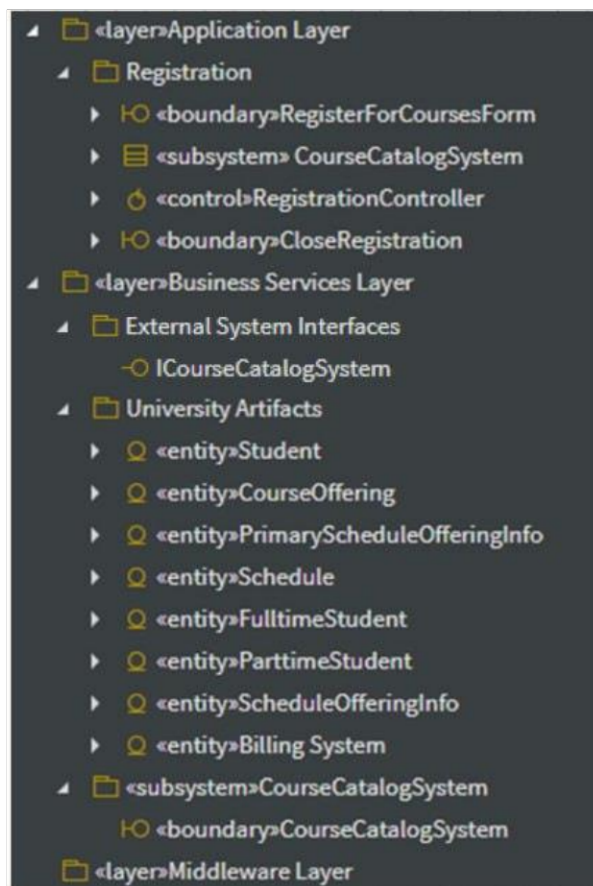


Рисунок 2.24 – Створена архітектура

Тепер необхідно створити діаграми. На рисунку 2.25 діаграма залежності між підсистемою та іншими пакетами (діаграма пакетів CourseCatalogSystem Dependencies), її необхідно створити у пакеті Business Services Layer.

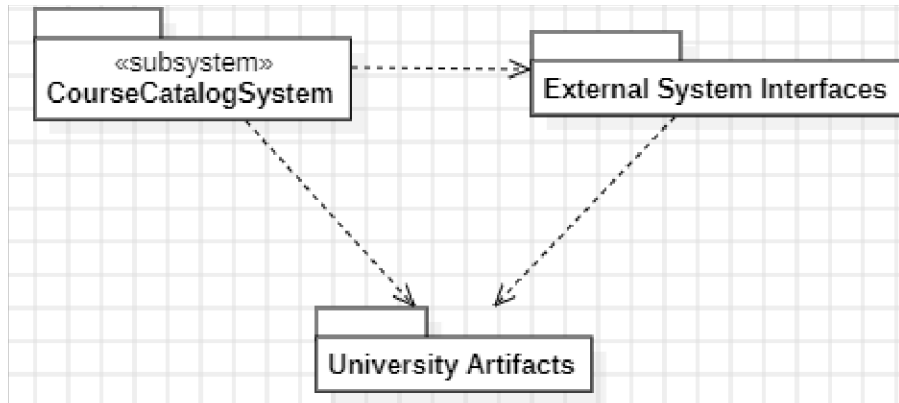


Рисунок 2.25 – Залежності між підсистемою та іншими пакетами (діаграма пакетів CourseCatalogSystem Dependencies)

Необхідно створити клас DBCourseOffering у пакеті University Artifacts, він відповідає за взаємодію з БД каталогу курсів (рис. 2.26).

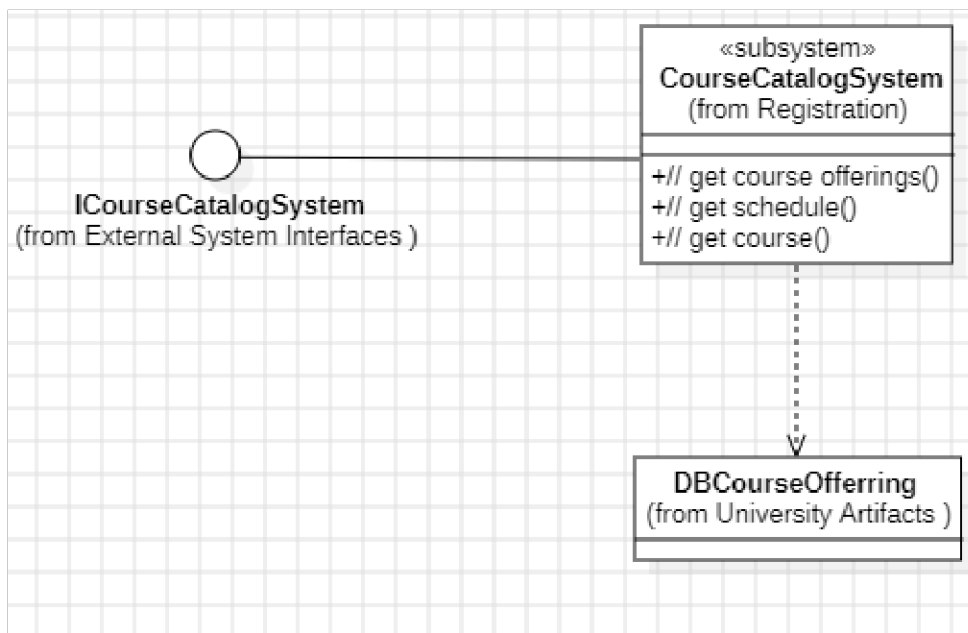


Рисунок 2.26 – Класи, що реалізують інтерфейс підсистеми (діаграма класів ICourseCatalogSystem)

Для наступної діаграми (рис. 2.27) необхідно створити класи CourseOfferingList та CourseCatalogSystemClient (зі стереотипами «entity») у пакеті University Artifacts. Діаграма має заходитись у пакеті External System Interfaces, як і наступна (рис. 2.28).

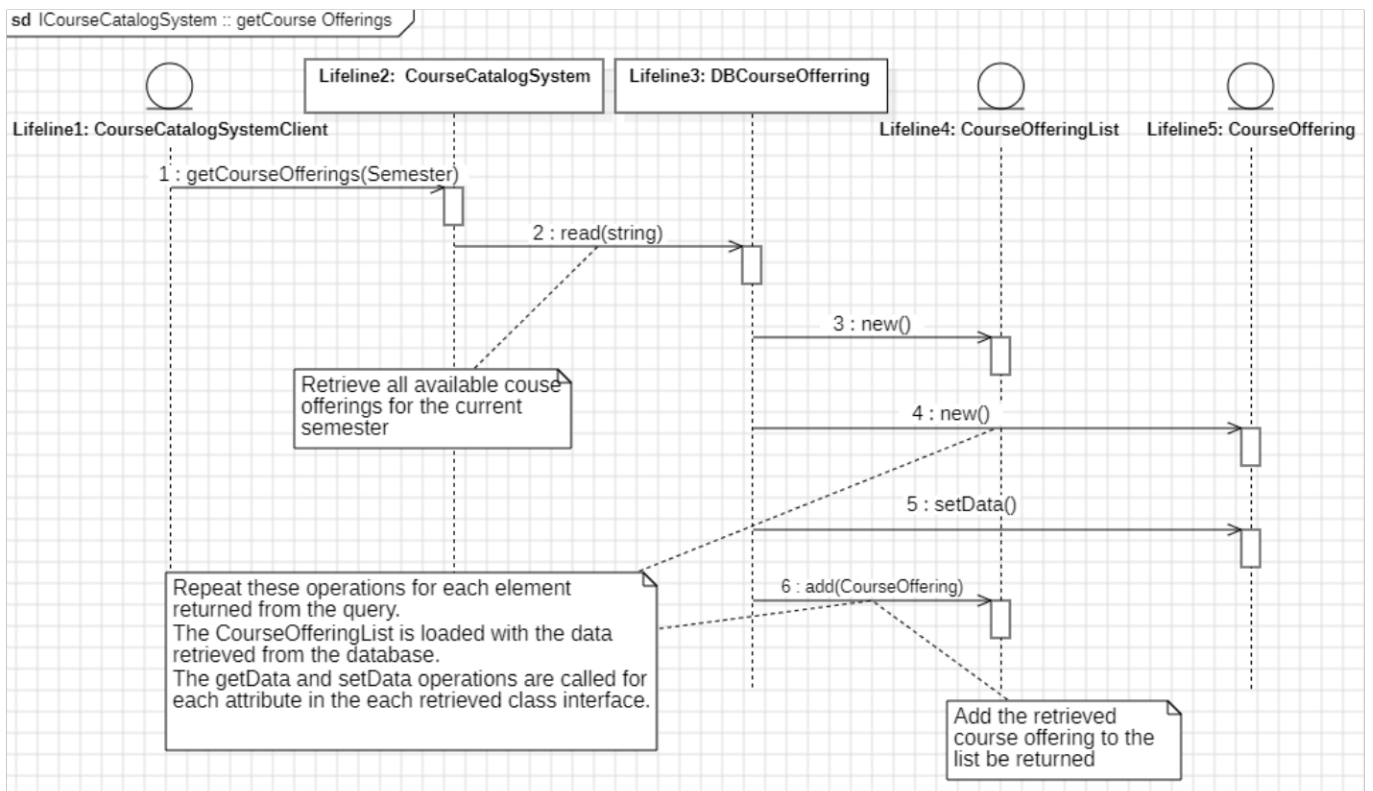


Рисунок 2.27 – Діаграма послідовності ICourseCatalogSystem :: getCourse Offerings, що описує взаємодію елементів при реалізації операції інтерфейсу getCourseOfferings.

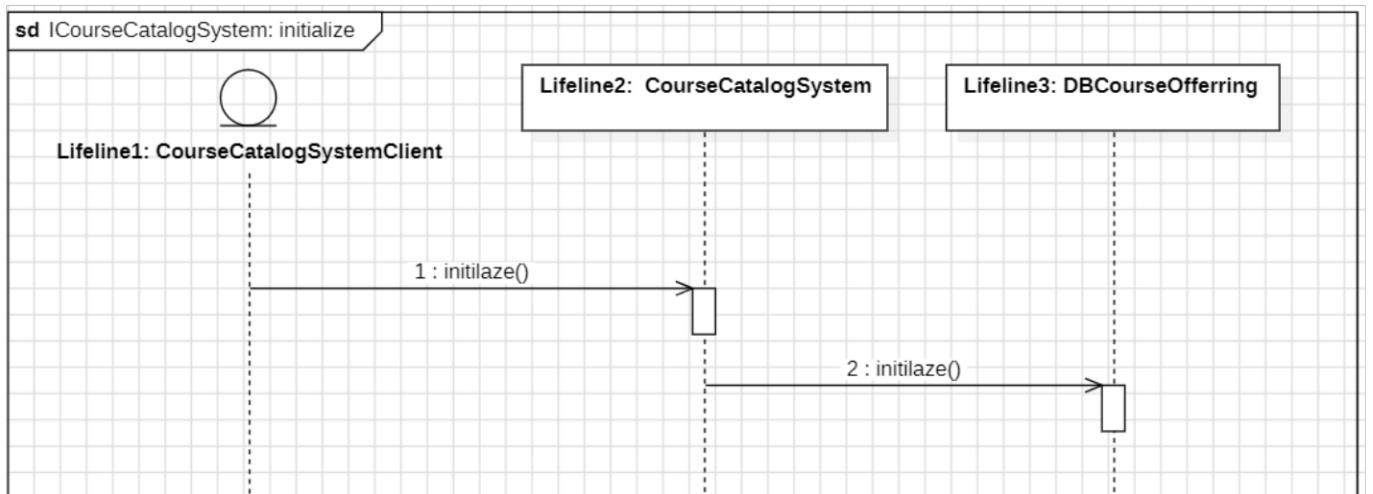


Рисунок 2.28 – Діаграма послідовності ICourseCatalogSystem: initialize, що описує взаємодію елементів при реалізації операції інтерфейсу initialize.

2. Моделювання розподіленої конфігурації системи

Розподілена конфігурація системи моделюється за допомогою діаграми розміщення. Її основні елементи:

- вузол (node) – обчислювальний ресурс (процесор, дискова пам'ять, контролери різних пристроїв і т. д.). Для вузла можна задати виконуються на ньому процеси;
- з'єднання (connection) – канал взаємодії вузлів (мережа). Приклад: мережева конфігурація системи реєстрації (без процесів).

Приклад: мережева конфігурація системи реєстрації (без процесів, рис. 2.29).

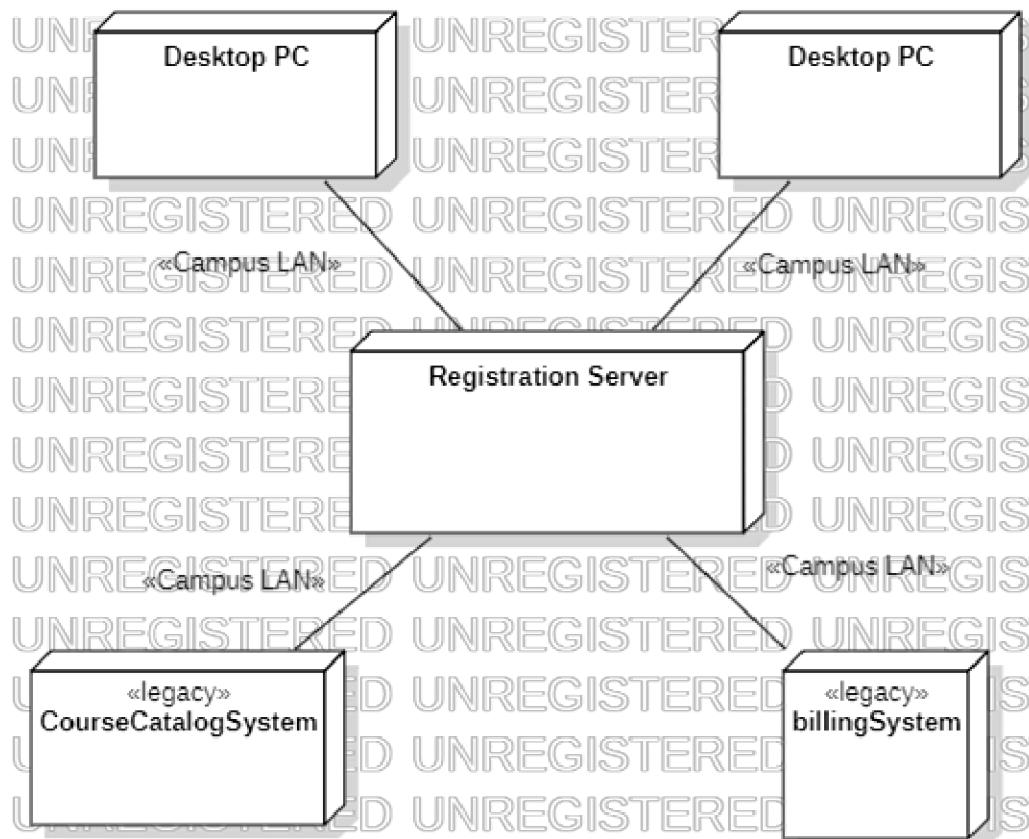


Рисунок 2.29 – Мережева конфігурація системи реєстрації

Розподіл процесів по вузлах мережі здійснюється з урахуванням таких факторів, як:

- використовувані зразки розподілу (трехзвінна клієнт-серверна конфігурація, «товстий клієнт», «тонкий клієнт», рівноправні вузли (peer-to-peer) і т.д.);

- час відгуку;
- мінімізація мережевого трафіку;
- потужність вузла;
- надійність устаткування і комунікацій.

Приклад розподілу процесів по вузлах наведено на рисунку 2.30.

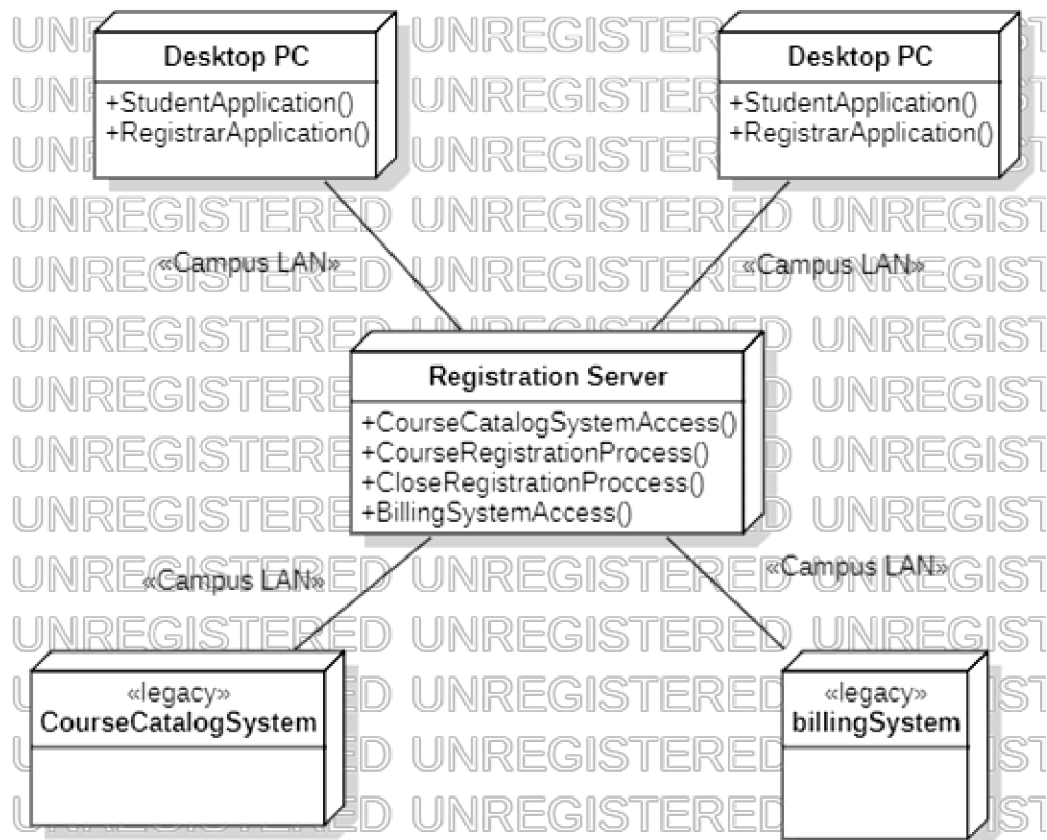


Рисунок 2.30 – Мережева конфігурація системи реєстрації з розподілом процесів по вузлах

Створення діаграми розміщення системи реєстрації. Щоб створити діаграму розміщення необхідно натиснути правою кнопкою миші по відповідному пакету Registration і обрати Add Diagram -> Deployment Diagram. Двічі клацніть лівою кнопкою миші по діаграмі, аби відкрити її.

Щоб помістити на діаграму вузол (процесор):

- 1) на панелі інструментів діаграми натисніть кнопку Node;
- 2) клацніть на діаграмі розміщення в тому місці, куди хочете його помістити;
- 3) введіть ім'я процесора;
- 4) поєднайте їх та задайте стереотип для зв'язків «Campus LAN»;
- 5) додайте методи, як на рисунку 2.30.

У специфікації процесора можна ввести інформацію про його стереотипі, характеристики. Стереотипи застосовуються для класифікації процесорів (наприклад, комп'ютерів під управлінням UNIX).

Характеристики процесора – це його фізичний опис. Воно може, зокрема, включати швидкість процесора і об'єм пам'яті.

Поле планування (scheduling) процесора містить опис того, як здійснюються планування його процесів:

- 1) preemptive (з пріоритетом). Високопріоритетні процеси мають перевагу перед низькопріоритетними;
- 2) non preemptive (без пріоритету) – у процесів немає пріоритету. Поточний процес виконується до його завершення, після чого починається наступний;
- 3) cyclic (циклічний) – управління передається між процесами по колу. Кожному процесу дається певний час на його виконання, потім управління переходить до наступного процесу;

4) executive (виконавчий) – існує деякий обчислювальний алгоритм, який і керує плануванням процесів;

5) manual (вручну) – процеси плануються користувачем.

Щоб призначити процесору стереотип:

1) натисніть лівою кнопкою по процесору;

2) знайдіть поле Stereotype і введіть до нього стереотип.

Щоб ввести характеристики і планування процесора:

1) відкрийте вікно специфікації процесора;

2) перейдіть на вкладку Detail;

3) введіть характеристики в поле характеристик;

4) вкажіть один з типів планування.

Щоб показати планування на діаграмі:

1) клацніть правою кнопкою миші на процесорі;

2) в меню, виберіть пункт Show Scheduling.

Щоб додати зв'язок на діаграму:

1) на панелі інструментів натисніть кнопку Communication Path;

2) клацніть на вузлі діаграми;

3) проведіть лінію зв'язку до іншого вузла.

Щоб призначити зв'язку стереотип:

1) відкрийте вікно специфікації зв'язку;

2) натисніть лівою кнопкою по зв'язку;

3) введіть стереотип в полі Stereotype.

Щоб додати процес:

1) клацніть двічі лівою кнопкою миші по вузлу;

2) в меню, виберіть пункт меню Add -> Operation;

3) введіть ім'я нового процесу.

Щоб показати процеси на діаграмі:

1) клацніть правою кнопкою миші на процесорі;

2) оберіть Add Operation (другий квадрат з правої сторони, іконка у вигляді шестерні).

3 Проектування класів

Класи аналізу перетворюються в проектні класи:

- проектування граничних класів – залежить від можливостей середовища розробки користувацького інтерфейсу (GUI Builder);

- проектування класів-сутностей – з урахуванням міркувань продуктивності (виділення в окремі-ні класи атрибутів з різною частотою використання);

- проектування керуючих класів – видалення класів, що реалізують просту передачу інформації від граничних класів до сутностям;

- ідентифікація стійких (persistent) класів, що містять збережену інформацію.

Обов'язки класів, визначені в процесі аналізу, перетворюються в операції. Кожній операції присвоюється ім'я, яке характеризує її результат. Визначається повна сигнатура операції: operationName (parameter: class, ...): returnType. Створюється короткий опис операції, включаючи сенс всіх її параметрів. Визначається видимість операції: public, private, protected. Визначається зона дії (scope) операції: екземпляр або класифікатор.

Визначаються (уточнюються) атрибути класів:

- крім імені, задається тип і значення за замовчуванням (необов'язкове):
attributeName: Type = Default;
- враховуються угоди по іменування атрибутів, прийняті в проєкті і мовою реалізації;
- задається видимість атрибутів: public, private, protected;
- при необхідності визначаються похідні (обчислювані) атрибути.

Визначення атрибутів і операцій для класу Student. Визначимо атрибути і операції для класу Student (рисунок 2.31).

Щоб задати тип даних, значення за замовчуванням і видимість атрибута:

- 1) клацніть лівою кнопкою миші по атрибуту в браузері чи на діаграмі;
- 2) знайдіть пункт visibility;
- 3) оберіть один з чотирьох можливих варіантів видимості атрибуту (за замовчуванням видимість всіх атрибутів відповідає public);
- 4) у полі type введіть тип даних атрибуту;
- 5) у полі defaultValue вкажіть значення атрибуту за замовчуванням.

Щоб змінити нотацію для позначення видимості:

- 1) оберіть елемент, для якого хочете змінити нотацію;
- 2) знайдіть поле Format;
- 3) оберіть один з варіантів у випадаючому списку (на рис. 2.31 – Icon).

Примітка. Зміна значення цього параметра приведе до зміни нотації тільки для нових діаграм і не торкнеться вже існуючих діаграм.

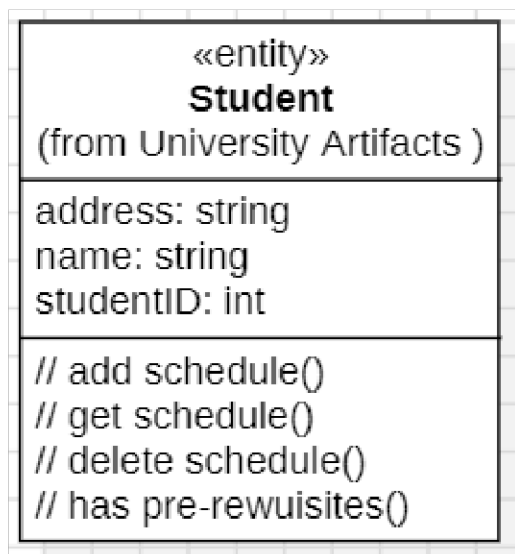


Рисунок 2.31 – Клас Student з повністю певними операціями та атрибутами

Щоб задати тип значення, що повертається, стереотип і видимість операції:

- 1) клацніть правою кнопкою миші на операції в браузері;
- 2) відкрийте вікно специфікації класу цієї операції;
- 3) вкажіть тип значення, що повертається в списку, що розкривається, або введіть свій тип;
- 4) вкажіть стереотип у полі stereotype або введіть новий;
- 5) у полі visibility вкажіть видимість операції.

Щоб додати до операції аргумент:

- 1) двічі клацніть лівою кнопкою миші по операції на діаграмі;

2) введіть тип аргументу у дужки.

Визначення станів для класів

Визначення станів для класів моделюється за допомогою **діаграм станів**. Діаграми станів створюються для опису об'єктів з високим рівнем динамічної поведінки.

Створення діаграми станів для класу CourseOffering. В якості прикладу розглянемо поведінку об'єкта – класу CourseOffering (рисунок 2.32). Він може знаходитися у відкритому стані (можливе додавання нового студента) або в закритому стані (максимальне кількість студентів вже записалося на курс).

Таким чином, конкретний стан залежить від кількості студентів, пов'язаних з об'єктом CourseOffering. Розглядаючи кожен варіант використання, можна виділити ще два стани: *ініціалізація* (до початку реєстрації студентів на курс) та *відміна* (курс виключається з розкладу).

Для створення діаграми станів для класу CourseOffering:

- 1) клацніть правою кнопкою миші в браузері на потрібному класі;
- 2) в меню, виберіть пункт меню Add Diagram -> Statechart Diagram.

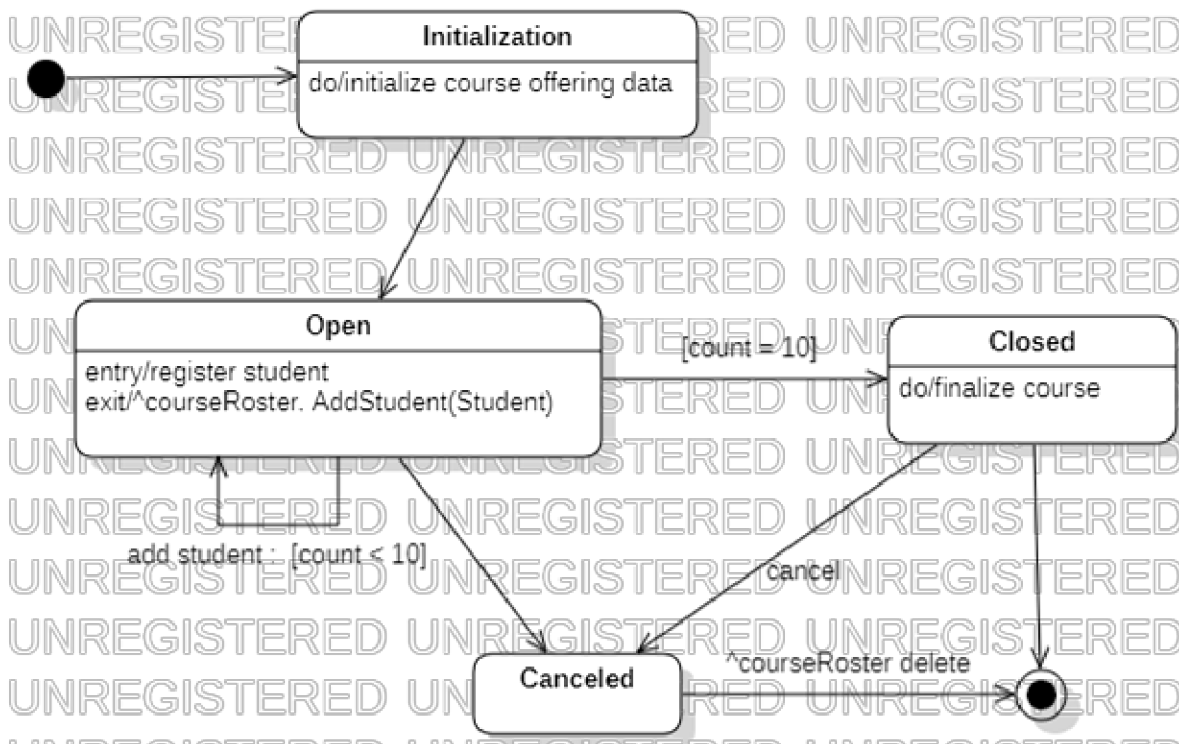


Рисунок 2.32 – Діаграма станів для класу CourseOffering

Щоб додати стан:

- 1) на панелі інструментів натисніть кнопку Simple State;
- 2) клацніть мишею на діаграмі станів в тому місці, куди треба його помістити.

Щоб додати діяльність:

- 1) клацніть правою кнопкою миші по стану на діаграмі;
- 2) якщо потрібно додати вхідну діяльність, то обираємо Add > Entry Activity (перший квадрат з правої сторони), щоб додати внутрішню діяльність – обираємо Add > Do Activity (другий квадрат), вихідну діяльність – Add > Exit Activity (третій квадрат).

Наступні дії можна виконувати за інструкцією нижче або вручну додавати

елементи з панелі Toolbox.

Щоб додати перехід:

- 1) натисніть кнопку Transition панелі інструментів;
- 2) клацніть мишею на стані, звідки здійснюється перехід;
- 3) проведіть лінію переходу до того стану, де він завершується.

Щоб послати подію:

- 1) двічі клацніть лівою кнопкою миші по стану на діаграмі;
- 2) обираємо Add Outgoing Transition (з правої сторони, другий рядок, перший квадрат).

Щоб додати рефлексивний перехід:

- 1) натисніть кнопку Self Transition панелі інструментів;
- 2) клацніть на тому стані, де здійснюється рефлексивний перехід.

Щоб додати подію, сторожову умову і дію:

- 1) клацніть на переході;
- 2) введіть назву події в поле name;
- 3) введіть сторожову умову в поле guard;

Щоб відправити подію:

- 1) двічі клацніть на переході, щоб відкрити вікно його специфікації;
- 2) перейдіть на вкладку Detail;
- 3) введіть подія в полі Send Event;
- 4) введіть аргументи на поле Send Arguments;
- 5) задайте мета в полі Send Target.

Для вказівки початкового або кінцевого стану:

- 1) на панелі інструментів натисніть кнопку Initial State або Final State;
- 2) клацніть мишею на діаграмі станів в тому місці, куди хочете помістити стан.

Уточнення асоціацій. Далі виконується уточнення асоціацій, тому деякі асоціації (семантичні, структурні, стійкі зв'язки за даними) можуть бути перетворені в залежності (неструктурні, тимчасові зв'язки, відображають видимість), а агрегації – в композиції.

Щоб встановити перетворення агрегації в композицію:

- 1) клацніть лівою кнопкою миші по зв'язку агрегації на діаграмі (см рис. 2.33 – Schedule);
- 2) знайдіть поле end (1 – те, звідки зв'язок виходить, 2 – куди зв'язок входить).aggregation;
- 3) оберіть composite.

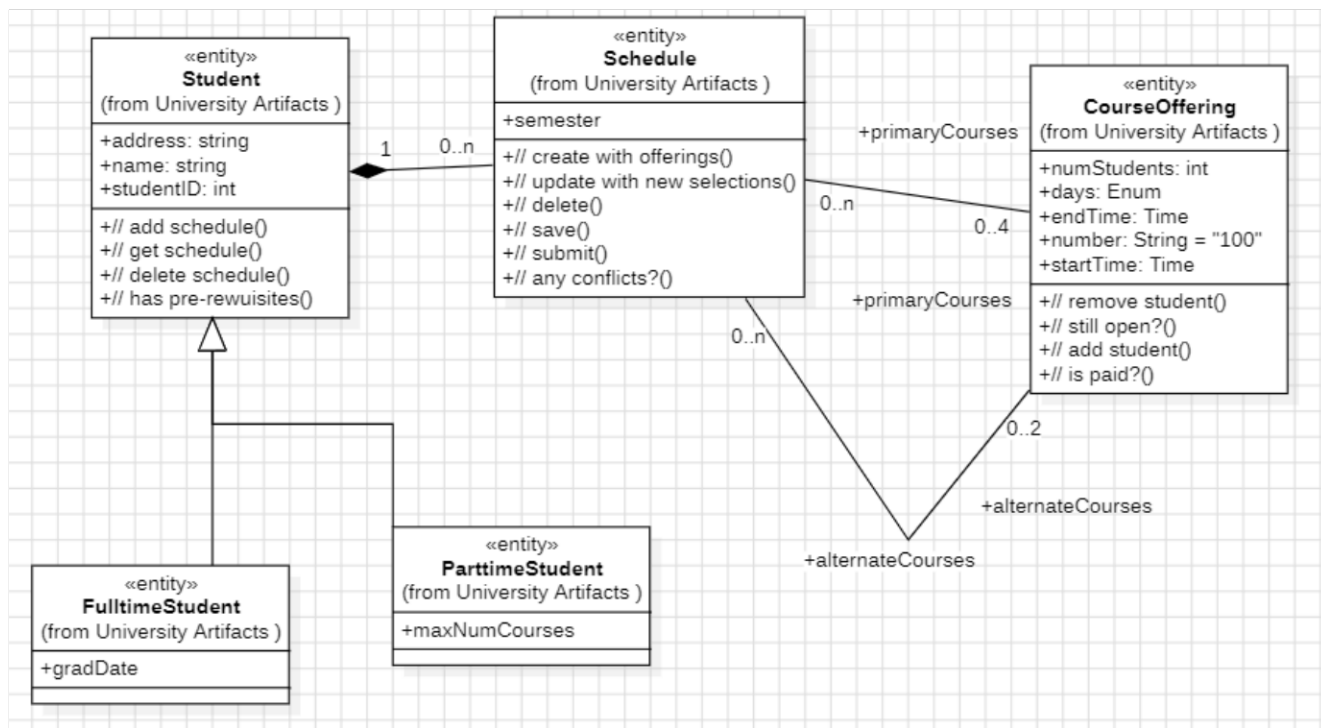


Рисунок 2.33 – Приклад перетворення асоціацій та агрегації

Майте на увазі, що значення composite припускає, що ціле і частина створюються і руйнуються одночасно, що відповідає композиції. Агрегація припускає, що ціле і частина створюються і руйнуються в різний час.

Уточнення узагальнень: в разі ситуації з міграцією підкласів (студент може переходити з очної форми навчання на вечірню) ієрархія спадкоємства реалізується так, як показано на рисунку 2.34. Таке рішення підвищує стійкість системи (не потрібно модифікувати опис об'єкта).

Необхідно створити нову діаграму (рис. 2.34) для перетворення узагальнення у пакеті Business Services Layer. Для неї необхідно створити клас Classification у пакеті University Artifacts.

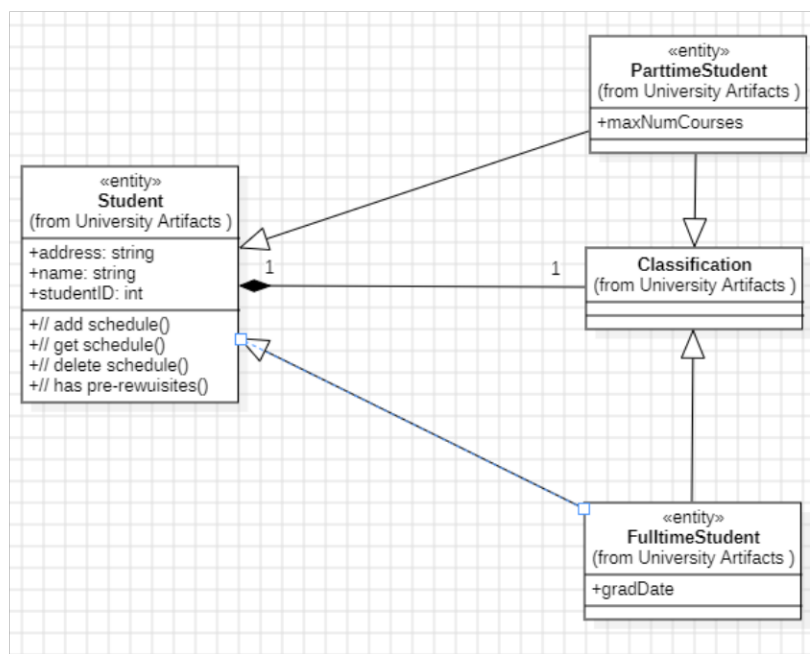


Рисунок 2.34 – Перетворення узагальнення

Фінальне дерево роботи має вигляд як на рисунку 2.35.

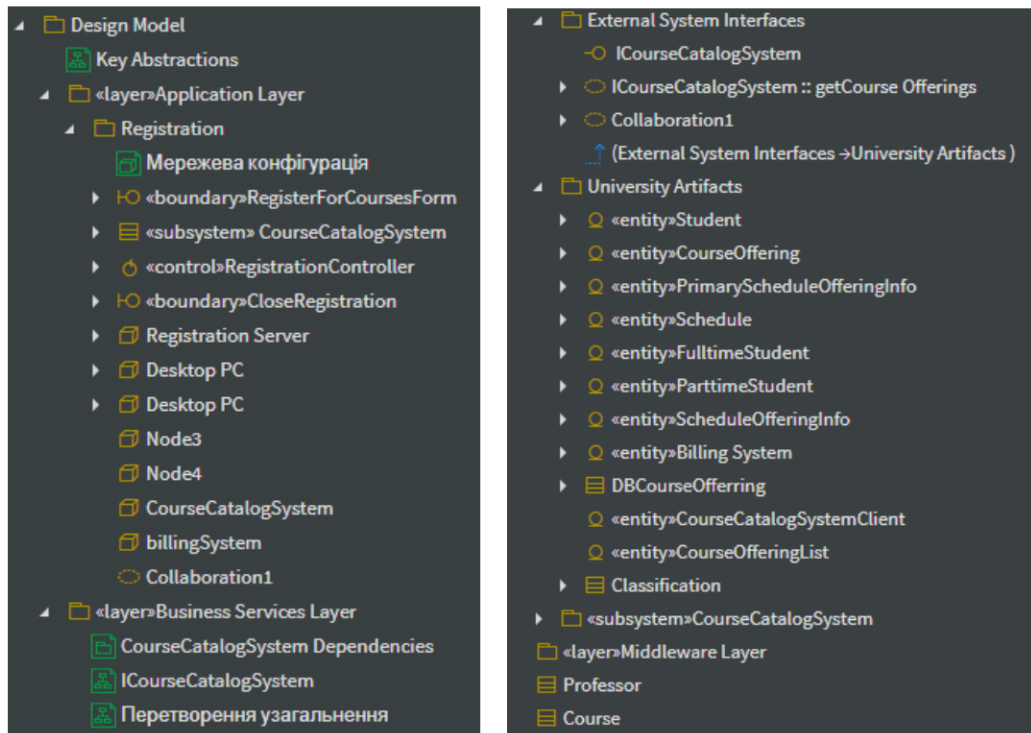


Рисунок 2.35 – Вигляд дерева